

Teoría de Conservación de Óptimos y Complejidad

Oscar Riveros

7 de abril de 2026

Resumen

Se formaliza una teoría local para estudiar cuándo una representación geométrica o alternativa de una familia de problemas de optimización discreta preserva simultáneamente: (i) el óptimo combinatorio y (ii) la complejidad temporal polinomial. La presente versión corrige los puntos débiles de los borradores previos separando con rigor cinco capas: codificación y tamaño de entrada, representación exacta, transporte algorítmico, equivalencia afín uniforme y obstrucciones formales. Se prueban como resultados centrales: conservación exacta del conjunto de minimizadores, transferencia de complejidad bajo exactitud fuerte, transporte de óptimos entre representaciones afínmente equivalentes y un criterio lineal necesario y suficiente para la existencia de levantamientos exactos de costos. Además se introduce una formulación correctamente tipada del cambio de representación para optimización, en la que la decodificación de soluciones, la recodificación final y el crecimiento representacional aparecen como parte explícita de la cota temporal.

También se aísla el contenido metamatemático del programa: para convertir una afirmación del tipo “una nueva representación geométrica de un problema NP-completo implica $P = NP$ ” en un teorema válido, no basta exhibir una intuición poliedral parcial, ni siquiera una condición de membresía eficiente; hace falta cerrar una cadena completa de hipótesis de representación exacta, fidelidad polinomial, extracción fuerte y puente formal hacia optimización exacta. Esa cadena se formula aquí como esquema suficiente explícito.

Índice

1. Alcance, contrato de rigor y regularización del borrador	2
2. Codificación binaria y modelos de representación	3
3. Familias de optimización lineal discreta	4
4. Representaciones exactas	5
5. Conservación del óptimo y transferencia de complejidad	6
6. Equivalencia afín entre representaciones	8
7. Criterio lineal para la existencia del levantamiento de costos	9
8. Cambio de representación y transporte de complejidad	11
9. Esquema suficiente hacia $P = NP$ y localización de los huecos	16

10.Obstrucciones formales	18
11.Programa matemático y problemas abiertos	19
12.Conclusión	20

Investigación relacionada. Una línea complementaria de investigación vinculada a este programa formal se encuentra en el repositorio *General Epistemic Geometry*: <https://github.com/maxtuno/General-Epistemic-Geometry>.

1. Alcance, contrato de rigor y regularización del borrador

La tesis central es aislar las condiciones exactas bajo las cuales la implicación

optimizar sobre una representación $\implies$ resolver el problema original en tiempo polinomial

es matemáticamente válida.

La regularización adoptada en esta versión sigue las reglas de evidencia siguientes.

- R1. Codificación explícita.** Toda afirmación de complejidad se hace respecto de una codificación binaria concreta o de un modelo de representación claramente tipado.
- R2. Separación entre existencia algebraica y efectividad algorítmica.** Que exista un levantamiento lineal de costos no implica por sí solo que dicho levantamiento sea computable en tiempo polinomial respecto del tamaño de la entrada.
- R3. Separación entre valor óptimo y solución óptima.** El soporte óptimo correcto conserva el valor óptimo; para extraer una solución combinatoria óptima a partir de un óptimo de la relajación se necesita un decodificador efectivo adicional.
- R4. Control del crecimiento representacional.** La exactitud semántica no transporta automáticamente la complejidad si el tamaño de la representación crece demasiado rápido.
- R5. Restricción del alcance.** Toda necesidad o suficiencia demostrada en este manuscrito debe leerse dentro del marco formal aquí fijado. No se afirma una caracterización universal de todas las posibles reducciones entre problemas de optimización.

Para volver completamente preciso el programa, fijamos además las normalizaciones estructurales siguientes.

- N1.** Los espacios de costos se toman como espacios vectoriales racionales finito-dimensionales. Esto vuelve literal la expresión “transformación lineal de costos”.
- N2.** Los conjuntos combinatorios S_n se suponen finitos y no vacíos. Con ello, el problema $\arg \min_{s \in S_n}$ queda siempre bien definido.
- N3.** Los poliedros de representación Q_n se tratan como politopos racionales en $\mathbb{R}^{d(n)}$, no como subconjuntos de $\mathbb{Q}^{d(n)}$. Sus vértices representables sí son racionales.
- N4.** La noción de exactitud fuerte se formula sobre entradas racionalmente codificadas. Esto explicita cómo entra en la complejidad el punto óptimo x^* sobre el que actúa el extractor.

N5. Las afirmaciones de transporte entre representaciones se formulan sobre problemas de optimización tipados, de modo que la salida del algoritmo, la inversión de la codificación de soluciones y la recodificación final queden bien distinguidas.

La teoría resultante distingue con claridad: equivalencia semántica entre soluciones, conservación exacta del óptimo, preservación de complejidad, obstrucciones formales al puente entre representaciones y límites de constructividad del criterio lineal cuando $|S_n|$ puede crecer exponencialmente.

2. Codificación binaria y modelos de representación

Definición 2.1 (Longitud binaria racional y de tuplas). Sea $q \in \mathbb{Q}$. Escribimos $q = a/b$ en forma reducida, con $b > 0$ y $\gcd(a, b) = 1$. Definimos su *longitud binaria* por

$$\langle q \rangle := \lceil \log_2(|a| + 1) \rceil + \lceil \log_2(b + 1) \rceil.$$

Para $n \in \mathbb{N}$, definimos

$$\langle n \rangle := \lceil \log_2(n + 1) \rceil.$$

Para un vector racional $u = (u_1, \dots, u_m) \in \mathbb{Q}^m$, definimos

$$\langle u \rangle := \sum_{i=1}^m \langle u_i \rangle.$$

Más generalmente, para una tupla finita de datos racionalmente codificados $(u_1, \dots, u_t)$, definimos

$$\langle u_1, \dots, u_t \rangle := \sum_{j=1}^t \langle u_j \rangle.$$

Observación 2.2. Cualquier otra codificación estándar equivalente por tiempo polinomial produciría una teoría idéntica hasta factores polinomiales. La elección anterior sólo fija una convención explícita para evitar ambigüedades.

Observación 2.3 (Separación de tipado entre racionales y cadenas). La notación $\langle \cdot \rangle$ se reserva en este manuscrito para objetos racionalmente codificados. Cuando el dato de entrada es una cadena binaria $x \in L_R$, su tamaño se escribe siempre como $|x|_R$, no como $\langle x \rangle$.

Definición 2.4 (Modelo estándar de representación). Un *modelo estándar de representación* es un par

$$R = (L_R, |\cdot|_R),$$

donde $L_R \subseteq \Sigma_R^*$ es un lenguaje de instancias sobre un alfabeto finito Σ_R , y $|x|_R$ es la longitud usual de la cadena $x \in L_R$.

Definición 2.5 (Envolvente de crecimiento). Sea $f : R_0 \rightarrow R_1$ una transformación entre modelos de representación. Definimos su *envolvente de crecimiento*

$$g_f(n) := \sup\{|f(x)|_{R_1} : x \in L_{R_0}, |x|_{R_0} \leq n\}.$$

Observación 2.6. En el modelo estándar de representación, el conjunto

$$\{x \in L_{R_0} : |x|_{R_0} \leq n\}$$

es finito para cada n , luego el supremo anterior es en realidad un máximo. El uso de $\sup$ mantiene, sin embargo, la definición correcta también cuando se describe un modelo de tamaño de manera más abstracta.

Definición 2.7 (Transformación afín estrictamente monótona). Una función $T : \mathbb{R} \rightarrow \mathbb{R}$ es una *transformación afín estrictamente monótona* si existen $a, b \in \mathbb{R}$ tales que

$$T(t) = at + b, \quad a > 0.$$

3. Familias de optimización lineal discreta

Definición 3.1 (Familia de optimización lineal discreta). Una *familia de optimización lineal discreta* es un sistema

$$\mathcal{P} = (S_n, C_n, \kappa_n)_{n \geq 1}$$

tal que, para cada $n \in \mathbb{N}$:

- (1) $S_n \neq \emptyset$ es un conjunto finito de soluciones combinatorias;
- (2) $C_n \leq \mathbb{Q}^{r(n)}$ es un subespacio vectorial racional de dimensión finita;
- (3) $\kappa_n : C_n \times S_n \rightarrow \mathbb{Q}$ satisface que, para todo $s \in S_n$, la aplicación

$$c \mapsto \kappa_n(c, s)$$

es lineal.

Observación 3.2 (Representación lineal concreta). Equivalente y concretamente, para cada $s \in S_n$ existe un vector $a_{n,s} \in \mathbb{Q}^{r(n)}$ tal que

$$\kappa_n(c, s) = \langle c, a_{n,s} \rangle.$$

En el TSP simétrico, puede tomarse

$$S_n = \{\text{tours hamiltonianos en } K_n\}, \quad C_n = \mathbb{Q}^{E(K_n)}, \quad \kappa_n(c, T) = c \cdot \chi_T,$$

donde $E(K_n)$ denota el conjunto de aristas del grafo completo K_n .

Observación 3.3 (Tamaño de instancia). Cuando hablemos del tiempo necesario para resolver una instancia $c \in C_n$, la medida de entrada será $\langle n, c \rangle$, salvo indicación expresa en contrario.

Definición 3.4 (Codificación combinatoria de soluciones). Una *codificación combinatoria de soluciones* para la familia $\mathcal{P}$ es una familia de inyecciones

$$\text{Enc}_n^S : S_n \hookrightarrow \{0, 1\}^*.$$

Su imagen

$$\text{Code}_n^S := \text{Im}(\text{Enc}_n^S) \subseteq \{0, 1\}^*$$

se interpreta como el *espacio concreto de códigos* de soluciones del nivel n . Se exige que la decodificación inversa sobre la imagen,

$$\text{Dec}_n^S := (\text{Enc}_n^S)^{-1} : \text{Code}_n^S \rightarrow S_n,$$

sea computable en tiempo polinomial en n y en la longitud del código de entrada.

Observación 3.5 (La codificación como interfaz de salida). La función Enc_n^S es aquí una estructura matemática que fija qué cadenas binarias codifican soluciones; su imagen Code_n^S es el objeto computacional primario. Para la teoría de complejidad desarrollada en este manuscrito no hace falta suponer que un algoritmo reciba como entrada una solución abstracta $s \in S_n$ y compute desde ella el código $\text{Enc}_n^S(s)$. Lo que sí hace falta es que la salida de un algoritmo pueda ser verificada y decodificada mediante Dec_n^S .

Definición 3.6 (Versiones codificadas de ϕ_n y ψ_n). Fijada una codificación combinatoria $(\text{Enc}_n^S)_n$, definimos las aplicaciones codificadas

$$\hat{\phi}_n := \phi_n \circ \text{Dec}_n^S : \text{Code}_n^S \rightarrow D_n, \quad \hat{\psi}_n := \text{Enc}_n^S \circ \psi_n : D_n \rightarrow \text{Code}_n^S.$$

Observación 3.7 (Tipado computacional de ϕ_n y ψ_n). Las afirmaciones de complejidad relevantes en este manuscrito recaen sobre $\widehat{\phi}_n$ y $\widehat{\psi}_n$, no sobre las aplicaciones abstractas $\phi_n : S_n \rightarrow D_n$ y $\psi_n : D_n \rightarrow S_n$ tomadas aisladamente como funciones entre objetos matemáticos.

Definición 3.8 (Resolución polinomial de la familia). Decimos que la familia $\mathcal{P}$ se *resuelve en tiempo polinomial* si existe una codificación combinatoria $(\text{Enc}_n^S)_n$ y un algoritmo que, dada la entrada binaria (n, c) , devuelve $\text{Enc}_n^S(s^*)$ para algún

$$s^* \in \arg \min_{s \in S_n} \kappa_n(c, s)$$

en tiempo polinomial en $\langle n, c \rangle$.

4. Representaciones exactas

Definición 4.1 (Representación exacta de optimización). Una *representación exacta de optimización* de $\mathcal{P}$ es una familia

$$\mathcal{R} = (Q_n, D_n, \phi_n, \psi_n, \Lambda_n, \beta_n)_{n \geq 1}$$

tal que, para cada n :

- (1) $Q_n \subseteq \mathbb{R}^{d(n)}$ es un politopo racional no vacío;
- (2) $D_n \subseteq \text{Vert}(Q_n)$ es el conjunto de vértices que representan soluciones válidas;
- (3) $\phi_n : S_n \rightarrow D_n$ es biyectiva;
- (4) $\psi_n : D_n \rightarrow S_n$ es la inversa de ϕ_n ;
- (5) $\Lambda_n : C_n \rightarrow \mathbb{Q}^{d(n)}$ es lineal;
- (6) $\beta_n : C_n \rightarrow \mathbb{Q}$ es lineal;
- (7) se cumple la *conservación exacta de valor*:

$$\kappa_n(c, s) = \Lambda_n(c) \cdot \phi_n(s) + \beta_n(c) \quad \forall c \in C_n, \forall s \in S_n.$$

Observación 4.2 (Por qué β_n puede tomarse lineal). Si en un borrador preliminar se escribe una corrección afín $c \mapsto \tilde{\beta}_n(c)$, entonces la linealidad de $c \mapsto \kappa_n(c, s)$ y de $c \mapsto \Lambda_n(c) \cdot \phi_n(s)$ fuerza $\tilde{\beta}_n(0) = 0$. Toda aplicación afín entre espacios vectoriales finito-dimensionales que anula al origen es lineal. Por eso la normalización lineal de β_n no pierde generalidad dentro del marco fijado.

Definición 4.3 (Soporte óptimo correcto). La representación $\mathcal{R}$ tiene *soporte óptimo correcto* si, para todo n y todo $c \in C_n$,

$$\arg \min_{x \in Q_n} \Lambda_n(c) \cdot x$$

contiene al menos un punto de D_n .

Definición 4.4 (Fidelidad polinomial). Fijada una codificación combinatoria $(\text{Enc}_n^S)_n$, la representación $\mathcal{R}$ es *polinomialmente fiel respecto de* $(\text{Enc}_n^S)_n$ si, además, se cumplen las condiciones siguientes:

- (1) las aplicaciones codificadas $\widehat{\phi}_n, \widehat{\psi}_n$, así como Λ_n y β_n , son computables en tiempo polinomial respecto del tamaño natural de sus entradas;
- (2) el tamaño binario de sus salidas está acotado por un polinomio en el tamaño binario de la entrada;

(3) se satisface una de las dos alternativas siguientes:

- (a) existe un algoritmo uniforme que, dada la entrada binaria n , genera una descripción racional explícita de Q_n en tiempo polinomial en $\langle n \rangle$, y el tamaño binario de la descripción producida está acotado por un polinomio en $\langle n \rangle$;
- (b) Q_n dispone de un oráculo de separación o membresía ejecutable en tiempo polinomial con las cotas bit-complejas usuales.

Observación 4.5. La fidelidad polinomial ya no se formula como computabilidad sobre soluciones abstractas $s \in S_n$, sino sobre la interfaz codificada Code_n^S introducida en la Definición 3.6. Ésta es la forma correcta de tipar los algoritmos. Además, la rama explícita del ítem (3a) se interpreta en un modelo uniforme binario: la descripción de Q_n debe poder *generarse* a partir de n en tiempo polinomial en $\langle n \rangle$, no simplemente existir como objeto de tamaño $\text{poly}(n)$ fuera del costo.

Definición 4.6 (Exactitud fuerte). La representación $\mathcal{R}$ es *fuertemente exacta* si tiene soporte óptimo correcto y, además, existe un algoritmo polinomial E_n tal que, dado $c \in C_n$ y dada la codificación racional de cualquier punto óptimo racional

$$x^* \in Q_n \cap \mathbb{Q}^{d(n)}$$

para la función objetivo $x \mapsto \Lambda_n(c) \cdot x$, produce un vértice óptimo

$$v^* = E_n(c, x^*) \in D_n$$

en tiempo polinomial en $\langle n, c, x^* \rangle$.

Observación 4.7 (Clase de salida admitida). La hipótesis anterior no exige decodificar todo punto real óptimo de Q_n , lo cual sería ajeno a un modelo binario estándar. Sólo exige decodificación sobre puntos óptimos racionalmente codificados, que son precisamente los objetos que pueden aparecer como salida de algoritmos exactos habituales sobre politopos racionales.

5. Conservación del óptimo y transferencia de complejidad

Teorema 5.1 (Conservación exacta del arg mín). Sea $\mathcal{R}$ una representación exacta de optimización con soporte óptimo correcto. Entonces, para todo n y todo $c \in C_n$,

$$s^* \in \arg \min_{s \in S_n} \kappa_n(c, s) \iff \phi_n(s^*) \in \arg \min_{x \in D_n} \Lambda_n(c) \cdot x.$$

Además,

$$\min_{x \in Q_n} \Lambda_n(c) \cdot x = \min_{x \in D_n} \Lambda_n(c) \cdot x.$$

Demostración. Por conservación exacta de valor,

$$\kappa_n(c, s) = \Lambda_n(c) \cdot \phi_n(s) + \beta_n(c).$$

Como $\beta_n(c)$ no depende de s , el orden de valores de $\kappa_n(c, \cdot)$ coincide exactamente con el orden de valores de $s \mapsto \Lambda_n(c) \cdot \phi_n(s)$. Dado que ϕ_n es biyectiva entre S_n y D_n , esto prueba la equivalencia de minimizadores sobre S_n y D_n . La igualdad entre los mínimos sobre Q_n y D_n es precisamente la hipótesis de soporte óptimo correcto. $\square$

Corolario 5.2 (Conservación del valor óptimo). Bajo las hipótesis del Teorema 5.1,

$$\min_{s \in S_n} \kappa_n(c, s) = \min_{x \in Q_n} \Lambda_n(c) \cdot x + \beta_n(c).$$

Demostración. Por el Teorema 5.1,

$$\min_{s \in S_n} \kappa_n(c, s) = \min_{x \in D_n} (\Lambda_n(c) \cdot x + \beta_n(c)) = \min_{x \in Q_n} \Lambda_n(c) \cdot x + \beta_n(c).$$

□

Teorema 5.3 (Transferencia de complejidad bajo exactitud fuerte). *Sea $(\text{Enc}_n^S)_{n \geq 1}$ una codificación combinatoria de soluciones de $\mathcal{P}$ en el sentido de la Definición 3.4. Sea $\mathcal{R}$ una representación exacta, polinomialmente fiel respecto de $(\text{Enc}_n^S)_n$ y fuertemente exacta de $\mathcal{P}$. Supóngase que existe un algoritmo que, dado n y dado $w \in \mathbb{Q}^{d(n)}$, computa en tiempo polinomial en $\langle n, w \rangle$ un punto óptimo racional*

$$x^* \in Q_n \cap \mathbb{Q}^{d(n)}$$

de

$$\min_{x \in Q_n} w \cdot x.$$

Entonces la familia $\mathcal{P}$ se resuelve en tiempo polinomial.

Demostración. Fijada la codificación combinatoria $(\text{Enc}_n^S)_{n \geq 1}$, dada una instancia $c \in C_n$, procedemos así:

- (1) computamos $w = \Lambda_n(c)$;
- (2) resolvemos $\min_{x \in Q_n} w \cdot x$ y obtenemos un óptimo racional $x^* \in Q_n$;
- (3) aplicamos el extractor fuerte y obtenemos $v^* = E_n(c, x^*) \in D_n$;
- (4) computamos directamente el código de salida

$$z^* = \hat{\psi}_n(v^*) = \text{Enc}_n^S(\psi_n(v^*)) \in \text{Code}_n^S$$

y devolvemos z^* .

Por la Definición 4.6, v^* es un vértice óptimo representable. Por el Teorema 5.1, la solución abstracta $\psi_n(v^*) \in S_n$ es óptima para el problema original, y por construcción $z^* = \text{Enc}_n^S(\psi_n(v^*))$ es precisamente su código binario de salida. Además, por fidelidad polinomial existe un polinomio p tal que

$$\langle w \rangle = \langle \Lambda_n(c) \rangle \leq p(\langle n, c \rangle),$$

de modo que la etapa (2) corre también en tiempo polinomial en $\langle n, c \rangle$. Cada etapa tiene entonces coste polinomial en $\langle n, c \rangle$: la primera por fidelidad polinomial, la segunda por la cota recién observada y por hipótesis, la tercera por exactitud fuerte y la cuarta por fidelidad polinomial de $\hat{\psi}_n$. La composición total es polinomial en el sentido de la Definición 3.8. □

Observación 5.4 (Lo que no da el soporte correcto). Sin exactitud fuerte sólo se conserva automáticamente el valor óptimo, como en el Corolario 5.2. La extracción efectiva de una solución combinatoria óptima a partir de un punto óptimo arbitrario de Q_n constituye una capa adicional y no una consecuencia automática de la mera representación exacta.

6. Equivalencia afín entre representaciones

Definición 6.1 (Familia afín racional uniformemente polinomialmente controlada). Sean $(U_n)_{n \geq 1}$ y $(V_n)_{n \geq 1}$ familias tales que

$$U_n \subseteq \mathbb{R}^{p(n)}, \quad V_n \subseteq \mathbb{R}^{q(n)}.$$

Diremos que una familia

$$(\tau_n : U_n \rightarrow V_n)_{n \geq 1}$$

es *afín racional uniformemente polinomialmente controlada* si, para cada n , existen una matriz $A_n \in \mathbb{Q}^{q(n) \times p(n)}$ y un vector $b_n \in \mathbb{Q}^{q(n)}$ tales que

$$\tau_n(x) = A_n x + b_n \quad \forall x \in U_n,$$

y además existe un único algoritmo uniforme Eval_τ y un único polinomio p_τ con las propiedades siguientes:

- (I) para toda entrada racional $x \in U_n \cap \mathbb{Q}^{p(n)}$, la salida $\tau_n(x)$ pertenece a $V_n \cap \mathbb{Q}^{q(n)}$;
- (II) dado (n, x) con $x \in U_n \cap \mathbb{Q}^{p(n)}$, el algoritmo Eval_τ calcula exactamente $\tau_n(x)$ en tiempo a lo sumo

$$p_\tau(\langle n, x \rangle + 1);$$

- (III) el tamaño binario de la salida satisface

$$\langle \tau_n(x) \rangle \leq p_\tau(\langle n, x \rangle + 1) \quad \forall x \in U_n \cap \mathbb{Q}^{p(n)}.$$

Observación 6.2. La uniformidad en n es indispensable. No basta con que cada τ_n sea computable por un polinomio dependiente de n ; para transportar complejidad a nivel de familias se necesita un algoritmo uniforme y una cota polinomial común.

Definición 6.3 (Equivalencia afín fuerte para optimización). Sean

$$\mathcal{R} = (Q_n, D_n, \phi_n, \psi_n, \Lambda_n, \beta_n)_{n \geq 1}, \quad \mathcal{R}' = (Q'_n, D'_n, \phi'_n, \psi'_n, \Lambda'_n, \beta'_n)_{n \geq 1}$$

dos representaciones exactas de la misma familia $\mathcal{P}$. Diremos que $\mathcal{R}$ y $\mathcal{R}'$ son *afínmente equivalentes para optimización* si existen familias de aplicaciones

$$\pi_n : Q_n \rightarrow Q'_n, \quad \iota_n : Q'_n \rightarrow Q_n$$

que son afines racionales uniformemente polinomialmente controladas en el sentido de la Definición 6.1, y tales que, para todo n :

- (1) $\pi_n(D_n) = D'_n$ e $\iota_n(D'_n) = D_n$;
- (2) $\pi_n \circ \iota_n = \text{id}$ en D'_n e $\iota_n \circ \pi_n = \text{id}$ en D_n ;
- (3) para todo $c \in C_n$, existe un escalar $\alpha_n(c) \in \mathbb{Q}$ tal que

$$\Lambda'_n(c) \cdot \pi_n(x) = \Lambda_n(c) \cdot x + \alpha_n(c) \quad \forall x \in Q_n,$$

y

$$\Lambda_n(c) \cdot \iota_n(x') = \Lambda'_n(c) \cdot x' - \alpha_n(c) \quad \forall x' \in Q'_n.$$

Observación 6.4. La compatibilidad de costos se exige sobre todo Q_n y todo Q'_n , no sólo sobre D_n y D'_n . Esa es la formulación adecuada cuando se quiere transportar puntos óptimos arbitrarios y, con ellos, exactitud fuerte.

Proposición 6.5 (Transporte de óptimos entre representaciones equivalentes). *Bajo las hipótesis de la Definición 6.3, para todo $c \in C_n$,*

$$x \in \arg \min_{u \in Q_n} \Lambda_n(c) \cdot u \implies \pi_n(x) \in \arg \min_{u' \in Q'_n} \Lambda'_n(c) \cdot u',$$

y simétricamente,

$$x' \in \arg \min_{u' \in Q'_n} \Lambda'_n(c) \cdot u' \implies \iota_n(x') \in \arg \min_{u \in Q_n} \Lambda_n(c) \cdot u.$$

Demostración. Sea x óptimo en Q_n . Para todo $y' \in Q'_n$,

$$\Lambda'_n(c) \cdot y' = \Lambda_n(c) \cdot \iota_n(y') + \alpha_n(c) \geq \Lambda_n(c) \cdot x + \alpha_n(c) = \Lambda'_n(c) \cdot \pi_n(x).$$

Por tanto $\pi_n(x)$ es óptimo en Q'_n . El argumento inverso es idéntico, usando la segunda identidad de la Definición 6.3. $\square$

Corolario 6.6 (Transferencia de soporte correcto y exactitud fuerte). *Si $\mathcal{R}$ y $\mathcal{R}'$ son afínmente equivalentes para optimización y $\mathcal{R}$ es fuertemente exacta, entonces $\mathcal{R}'$ también es fuertemente exacta. En particular, el soporte óptimo correcto se transporta de $\mathcal{R}$ a $\mathcal{R}'$.*

Demostración. Sea $x'^* \in Q'_n \cap \mathbb{Q}^{d'(n)}$ un óptimo racional para $\Lambda'_n(c)$. Como la familia $(\iota_n)_n$ es afín racional uniformemente polinomialmente controlada, el punto $\iota_n(x'^*)$ pertenece a $Q_n \cap \mathbb{Q}^{d(n)}$ y su codificación racional tiene tamaño polinomial en $\langle n, x'^* \rangle$. Por la Proposición 6.5, $\iota_n(x'^*)$ es óptimo para $\Lambda_n(c)$. Aplicando el extractor fuerte de $\mathcal{R}$, obtenemos $v^* \in D_n$ óptimo. Entonces $\pi_n(v^*) \in D'_n$ es óptimo en Q'_n , otra vez por la Proposición 6.5. Esto define un extractor fuerte para $\mathcal{R}'$:

$$E'_n(c, x'^*) := \pi_n(E_n(c, \iota_n(x'^*))).$$

El soporte óptimo correcto es el caso particular en que sólo importa la existencia de un vértice representable óptimo. $\square$

Observación 6.7 (Transporte de complejidad). La fidelidad polinomial no se transfiere automáticamente a partir de la mera equivalencia afín: también hay que controlar de manera uniforme en n el tamaño binario y el costo de evaluación de π_n, ι_n , así como la codificación de Q'_n y de los datos inducidos. Una vez impuestas esas cotas uniformes, el transporte de complejidad se obtiene por composición polinomial ordinaria.

7. Criterio lineal para la existencia del levantamiento de costos

Definición 7.1 (Matriz de representación y vector de costos). Fijado n , sea $S_n = \{s_1, \dots, s_N\}$. Definimos la matriz

$$M_n \in \mathbb{Q}^{N \times (d(n)+1)}$$

cuya fila i -ésima es

$$(1, \phi_n(s_i)).$$

Definimos además el mapa lineal de costos

$$K_n : C_n \rightarrow \mathbb{Q}^N, \quad K_n(c)_i = \kappa_n(c, s_i).$$

Teorema 7.2 (Criterio lineal de existencia). *Existen aplicaciones lineales $\Lambda_n : C_n \rightarrow \mathbb{Q}^{d(n)}$ y $\beta_n : C_n \rightarrow \mathbb{Q}$ tales que*

$$\kappa_n(c, s) = \Lambda_n(c) \cdot \phi_n(s) + \beta_n(c) \quad \forall c \in C_n, \forall s \in S_n,$$

si y sólo si

$$K_n(c) \in \text{Col}(M_n) \quad \forall c \in C_n.$$

Demostración. Sea $L_n : \mathbb{Q}^{d(n)+1} \rightarrow \mathbb{Q}^N$ la aplicación lineal $z \mapsto M_n z$. Si existen Λ_n y β_n , definimos

$$B_n(c) := \begin{pmatrix} \beta_n(c) \\ \Lambda_n(c) \end{pmatrix} \in \mathbb{Q}^{d(n)+1}.$$

Entonces

$$K_n(c) = L_n(B_n(c)) = M_n \begin{pmatrix} \beta_n(c) \\ \Lambda_n(c) \end{pmatrix},$$

luego $K_n(c) \in \text{Col}(M_n)$ para todo c .

Recíprocamente, supóngase que $K_n(C_n) \subseteq \text{Col}(M_n)$. Como $K_n(C_n)$ es un subespacio de $\text{Col}(M_n)$, existe una sección lineal

$$R_n : K_n(C_n) \rightarrow \mathbb{Q}^{d(n)+1}$$

de L_n , es decir,

$$L_n \circ R_n = \text{id}_{K_n(C_n)}.$$

Definimos entonces

$$B_n := R_n \circ K_n : C_n \rightarrow \mathbb{Q}^{d(n)+1}.$$

La aplicación B_n es lineal. Escribiendo

$$B_n(c) = \begin{pmatrix} \beta_n(c) \\ \Lambda_n(c) \end{pmatrix},$$

obtenemos

$$K_n(c) = L_n(B_n(c)) = M_n \begin{pmatrix} \beta_n(c) \\ \Lambda_n(c) \end{pmatrix},$$

y, al leer la coordenada i -ésima, se concluye

$$\kappa_n(c, s_i) = \Lambda_n(c) \cdot \phi_n(s_i) + \beta_n(c).$$

□

Corolario 7.3 (Criterio de refutación). *Si existe $c \in C_n$ tal que*

$$K_n(c) \notin \text{Col}(M_n),$$

entonces no existe levantamiento lineal exacto de costos para ese n .

Proposición 7.4 (Barrera de materialización explícita). *Sea $N = |S_n|$. Si el acceso algorítmico a $K_n(c) \in \mathbb{Q}^N$ y a $M_n \in \mathbb{Q}^{N \times (d(n)+1)}$ es únicamente mediante su materialización explícita coordenada a coordenada, entonces cualquier procedimiento que construya por completo alguno de esos objetos necesita tiempo y salida de tamaño $\Omega(N)$. En particular, del Teorema 7.2 no se deduce ninguna computabilidad polinomial en n cuando N puede crecer exponencialmente.*

Demostración. Tanto el vector $K_n(c)$ como la matriz M_n tienen N filas explícitas. Escribir completamente cualquiera de esos objetos requiere producir al menos N bloques de datos; por tanto el tiempo y el tamaño de salida son $\Omega(N)$. Si N es exponencial en n , esa sola etapa ya impide una implementación polinomial en n . □

Proposición 7.5 (Versión constructiva correcta bajo acceso implícito). *Supóngase que se satisface el criterio lineal del Teorema 7.2. Entonces el levantamiento exacto de costos es computable en tiempo polinomial en cualquiera de los regímenes siguientes:*

- (a) régimen explícito pequeño: $|S_n|$, $d(n)$ y $r(n)$ están acotados polinomialmente; existe una base efectiva de C_n ; S_n es enumerable en tiempo polinomial; y ϕ_n , κ_n son evaluables en tiempo polinomial;
- (b) régimen implícito/oracular: existe un procedimiento polinomial que, dado $c \in C_n$, computa directamente un vector

$$B_n(c) = \begin{pmatrix} \beta_n(c) \\ \Lambda_n(c) \end{pmatrix}$$

sin materializar M_n ni $K_n(c)$ completos.

Demostración. En el caso (a), todos los objetos relevantes tienen tamaño polinomial y pueden construirse explícitamente; por tanto se puede resolver el sistema lineal correspondiente y obtener $B_n(c)$ en tiempo polinomial. En el caso (b), la hipótesis ya proporciona directamente el procedimiento implícito necesario para computar el levantamiento. $\square$

Observación 7.6. La versión anterior, que pretendía deducir constructividad sólo a partir del criterio $K_n(C_n) \subseteq \text{Col}(M_n)$, era falsa en general: cuando $|S_n|$ es exponencial, la mera materialización de los datos puede destruir cualquier cota polinomial.

8. Cambio de representación y transporte de complejidad

Definición 8.1 (Problema de optimización tipado en una representación). Un *problema de optimización tipado* sobre un modelo estándar de representación R es una tupla

$$\mathcal{O}_R = (L_R, \text{Sol}_R, \text{Val}_R, \text{Enc}_R^{\text{sol}}, |\cdot|_R)$$

tal que:

- (1) para cada instancia $x \in L_R$, el conjunto $\text{Sol}_R(x)$ de soluciones válidas es finito y no vacío;
- (2) la función de valor

$$\text{Val}_R(x, \cdot) : \text{Sol}_R(x) \rightarrow \mathbb{Q}$$

está bien definida;

- (3) la aplicación

$$\text{Enc}_R^{\text{sol}}(x, \cdot) : \text{Sol}_R(x) \hookrightarrow \{0, 1\}^*$$

es inyectiva, y su imagen

$$\text{Code}_R(x) := \text{Im}(\text{Enc}_R^{\text{sol}}(x, \cdot)) \subseteq \{0, 1\}^*$$

se interpreta como el espacio concreto de códigos de soluciones;

- (4) la decodificación sobre la imagen,

$$\text{Dec}_R^{\text{sol}}(x, \cdot) := (\text{Enc}_R^{\text{sol}}(x, \cdot))^{-1} : \text{Code}_R(x) \rightarrow \text{Sol}_R(x),$$

está uniformemente bien definida y satisface que, para todo

$$z \in \text{Code}_R(x),$$

el valor $\text{Dec}_R^{\text{sol}}(x, z)$ es computable en tiempo polinomial en $|x|_R + |z|$.

Observación 8.2. La estructura computacional relevante entra por los códigos $z \in \text{Code}_R(x)$. No se exige aquí que un algoritmo reciba una solución abstracta $y \in \text{Sol}_R(x)$ como entrada primaria y compute desde ella $\text{Enc}_R^{\text{sol}}(x, y)$; la interfaz concreta la fija la imagen $\text{Code}_R(x)$ y su decodificación efectiva.

Definición 8.3 (Resolución en una representación). Un algoritmo *resuelve* $\mathcal{O}_R$ si, dada una instancia $x \in L_R$, produce una cadena

$$z^* \in \text{Code}_R(x)$$

tal que

$$\text{Dec}_R^{\text{sol}}(x, z^*) \in \arg \min_{y \in \text{Sol}_R(x)} \text{Val}_R(x, y).$$

Definición 8.4 (Transformación semánticamente fiel para optimización). Sean

$$\mathcal{O}_{R_0} = (L_{R_0}, \text{Sol}_{R_0}, \text{Val}_{R_0}, \text{Enc}_{R_0}^{\text{sol}}, |\cdot|_{R_0}),$$

$$\mathcal{O}_{R_1} = (L_{R_1}, \text{Sol}_{R_1}, \text{Val}_{R_1}, \text{Enc}_{R_1}^{\text{sol}}, |\cdot|_{R_1}).$$

Sea $f : L_{R_0} \rightarrow L_{R_1}$. Diremos que f es una *transformación semánticamente fiel para optimización* si existen datos uniformes con las propiedades siguientes:

- (1) f es computable;
- (2) para cada instancia $x \in L_{R_0}$, existen aplicaciones abstractas

$$\eta_x : \text{Sol}_{R_0}(x) \rightarrow \text{Sol}_{R_1}(f(x)), \quad \delta_x : \text{Sol}_{R_1}(f(x)) \rightarrow \text{Sol}_{R_0}(x),$$

y realizadores sobre códigos

$$\hat{\eta}_x : \text{Code}_{R_0}(x) \rightarrow \text{Code}_{R_1}(f(x)), \quad \hat{\delta}_x : \text{Code}_{R_1}(f(x)) \rightarrow \text{Code}_{R_0}(x),$$

tales que los diagramas de codificación/decodificación conmutan:

$$\text{Dec}_{R_1}^{\text{sol}}(f(x), \hat{\eta}_x(z)) = \eta_x(\text{Dec}_{R_0}^{\text{sol}}(x, z)) \quad \forall z \in \text{Code}_{R_0}(x),$$

$$\text{Dec}_{R_0}^{\text{sol}}(x, \hat{\delta}_x(w)) = \delta_x(\text{Dec}_{R_1}^{\text{sol}}(f(x), w)) \quad \forall w \in \text{Code}_{R_1}(f(x));$$

- (3) se cumple la ley de reidentificación en el origen:

$$\delta_x \circ \eta_x = \text{id}_{\text{Sol}_{R_0}(x)};$$

- (4) para cada $x \in L_{R_0}$ existe una transformación afín estrictamente monótona

$$T_x(t) = a_x t + b_x, \quad a_x > 0,$$

tal que la compatibilidad de valores vale para toda solución de la representación destino:

$$\text{Val}_{R_1}(f(x), y) = T_x(\text{Val}_{R_0}(x, \delta_x(y))) \quad \forall y \in \text{Sol}_{R_1}(f(x)).$$

Definición 8.5 (Transformación polinomialmente fiel para optimización). Una transformación semánticamente fiel f es *polinomialmente fiel* si, además, existen polinomios $p_f, p_g, p_{\hat{\eta}}, p_{\hat{\delta}}$ tales que:

- (1) el tiempo de computar $f(x)$ está acotado por $p_f(|x|_{R_0})$;
- (2) la envolvente de crecimiento satisface $g_f(n) \leq p_g(n)$;

(3) el tiempo de computar $\hat{\eta}_x(z)$ está acotado por

$$p_{\hat{\eta}}(|x|_{R_0} + |z|) \quad \forall z \in \text{Code}_{R_0}(x);$$

(4) el tiempo de computar $\hat{\delta}_x(w)$ está acotado por

$$p_{\hat{\delta}}(|x|_{R_0} + |w|) \quad \forall w \in \text{Code}_{R_1}(f(x)).$$

Observación 8.6. La capa computacional se fija ahora de manera no circular: los algoritmos actúan sobre cadenas z, w en los conjuntos de códigos $\text{Code}_{R_0}(x)$ y $\text{Code}_{R_1}(f(x))$, mientras que las funciones abstractas η_x, δ_x quedan como mapas semánticos inducidos.

Observación 8.7 (Variante más débil). Una variante más débil reemplaza la compatibilidad total del ítem (4) de la Definición 8.4 por compatibilidad sólo sobre la imagen de η_x . En ese caso, para cerrar el teorema de transporte, hay que añadir la hipótesis de que el algoritmo en la representación R_1 devuelve códigos dentro de $\text{Im}(\hat{\eta}_x)$, o equivalentemente soluciones dentro de $\text{Im}(\eta_x)$. La versión adoptada aquí evita esa cláusula adicional.

Observación 8.8. La cláusula de decodificación es estructural para optimización. Sin ella, un algoritmo que trabaja en R_1 puede producir un código perfectamente válido en la representación destino y , sin embargo, no producir un código interpretable en la representación fuente.

Definición 8.9 (Envolvente monótona de una cota temporal). Sea $T : \mathbb{N} \rightarrow \mathbb{N}$. Definimos

$$T^\uparrow(m) := \max\{T(t) : 0 \leq t \leq m\}.$$

Observación 8.10. La función $T^\uparrow$ es no decreciente y satisface $T(m) \leq T^\uparrow(m)$ para todo m . Si un algoritmo corre en tiempo a lo sumo $T(m)$ sobre entradas de tamaño m , entonces también corre en tiempo a lo sumo $T^\uparrow(m)$. Esto permite escribir las cotas de transporte sin imponer monotonía adicional sobre T .

Proposición 8.11 (Transporte directo para decisión o salida ya compatible). Sea $f : R_0 \rightarrow R_1$ computable en tiempo $F(n)$. Si un algoritmo A resuelve instancias en la representación R_1 en tiempo $T(m)$, y la salida de A ya es directamente válida para R_0 sin decodificación adicional, entonces el algoritmo inducido sobre R_0 corre en tiempo

$$F(n) + T^\uparrow(g_f(n)).$$

Demostración. Dada una instancia x con $|x|_{R_0} = n$, primero se calcula $f(x)$, lo cual cuesta $F(n)$. Por definición de g_f , la instancia resultante tiene tamaño a lo sumo $g_f(n)$. Como A corre en tiempo a lo sumo $T(m)$ sobre entradas de tamaño m , también corre en tiempo a lo sumo $T^\uparrow(g_f(n))$ sobre $f(x)$. No hay paso adicional de decodificación. $\square$

Lema 8.12 (La longitud de la salida no excede el tiempo). Sea A un algoritmo determinista que, sobre entradas de tamaño m , termina en tiempo $T(m)$ y produce una cadena binaria z . Entonces

$$|z| \leq T(m).$$

Demostración. Cada bit de salida requiere, como mínimo, una operación elemental de escritura. Por tanto, ningún algoritmo que termine tras $T(m)$ pasos puede imprimir más de $T(m)$ bits. $\square$

Teorema 8.13 (Transporte de complejidad por cambio de representación en optimización). Sea

$$f : \mathcal{O}_{R_0} \longrightarrow \mathcal{O}_{R_1}$$

una transformación semánticamente fiel para optimización. Supóngase que:

- (1) el tiempo de computar f sobre entradas x con $|x|_{R_0} = n$ está acotado por una función uniforme $F(n)$;
- (2) el tiempo de evaluar el realizador codificado

$$\widehat{\delta}_x(w) \in \text{Code}_{R_0}(x),$$

para instancias $x \in L_{R_0}$ con $|x|_{R_0} = n$ y códigos $w \in \text{Code}_{R_1}(f(x))$ de longitud a lo sumo ℓ , está acotado por una función uniforme $\widehat{D}(n, \ell)$.

Si un algoritmo A resuelve $\mathcal{O}_{R_1}$ en tiempo $T(m)$, entonces el algoritmo inducido que, sobre una instancia $x \in L_{R_0}$,

- (1) computa $f(x)$;
- (2) ejecuta A sobre $f(x)$ y obtiene un código

$$z^* \in \text{Code}_{R_1}(f(x));$$

- (3) transporta el código de salida por

$$z_0^* = \widehat{\delta}_x(z^*) \in \text{Code}_{R_0}(x);$$

- (4) devuelve z_0^* ,

corre en tiempo

$$F(n) + T^\uparrow(g_f(n)) + \widehat{D}(n, T^\uparrow(g_f(n))).$$

En particular, si f es polinomialmente fiel y T es polinomial, entonces el algoritmo inducido sobre R_0 es polinomial.

Demostración. Sea $x \in L_{R_0}$ y sea $z^* \in \text{Code}_{R_1}(f(x))$ la salida producida por A sobre $f(x)$. Por la Definición 8.3, el código z^* representa una solución óptima $y^* \in \text{Sol}_{R_1}(f(x))$, definida semánticamente por

$$y^* := \text{Dec}_{R_1}^{\text{sol}}(f(x), z^*) \in \arg \min_{y \in \text{Sol}_{R_1}(f(x))} \text{Val}_{R_1}(f(x), y).$$

Definimos ahora

$$z_0^* := \widehat{\delta}_x(z^*) \in \text{Code}_{R_0}(x), \quad s^* := \text{Dec}_{R_0}^{\text{sol}}(x, z_0^*) \in \text{Sol}_{R_0}(x).$$

Por conmutatividad del diagrama de codificación/decodificación,

$$s^* = \delta_x(y^*).$$

Debemos probar que s^* es óptima para x . Sea $s \in \text{Sol}_{R_0}(x)$ arbitraria. Como $\eta_x(s) \in \text{Sol}_{R_1}(f(x))$ y y^* es óptima en la representación destino, se tiene

$$\text{Val}_{R_1}(f(x), y^*) \leq \text{Val}_{R_1}(f(x), \eta_x(s)).$$

Aplicando la compatibilidad total de valores y la identidad $\delta_x \circ \eta_x = \text{id}$, obtenemos

$$T_x(\text{Val}_{R_0}(x, s^*)) = \text{Val}_{R_1}(f(x), y^*) \leq \text{Val}_{R_1}(f(x), \eta_x(s)) = T_x(\text{Val}_{R_0}(x, s)).$$

Como T_x es estrictamente monótona creciente, concluimos

$$\text{Val}_{R_0}(x, s^*) \leq \text{Val}_{R_0}(x, s) \quad \forall s \in \text{Sol}_{R_0}(x).$$

Por tanto s^* es óptima en R_0 .

Para la cota temporal, sea $n = |x|_{R_0}$. Computar $f(x)$ cuesta $F(n)$. Como A corre en tiempo a lo sumo $T(m)$ sobre entradas de tamaño m , también corre en tiempo a lo sumo $T^\uparrow(g_f(n))$ sobre $f(x)$. Por el Lema 8.12, la longitud del código producido satisface

$$|z^*| \leq T^\uparrow(g_f(n)).$$

Por definición de $\widehat{D}$, transportar el código z^* hacia z_0^* cuesta a lo sumo $\widehat{D}(n, T^\uparrow(g_f(n)))$. Sumando las etapas se obtiene la cota anunciada. $\square$

Corolario 8.14 (Preservación polinomial). *Si f es polinomialmente fiel y el algoritmo A sobre R_1 es polinomial, entonces el algoritmo inducido por el Teorema 8.13 es polinomial.*

Demostración. Si A es polinomial, existe un polinomio no decreciente p tal que su tiempo de ejecución está acotado por $p(m)$. Entonces $T^\uparrow(m) \leq p(m)$ para todo m , y basta componer cotas polinomiales uniformes. $\square$

Observación 8.15 (Advertencia metodológica bajo expansión exponencial). Supóngase que existe $c > 1$ tal que $g_f(n) \geq c^n$ para infinitos n . Entonces la sola fidelidad semántica del cambio de representación no basta, por sí misma, para inferir preservación polinomial a partir del Teorema 8.13. En efecto, la cota de transporte contiene la composición $T^\uparrow(g_f(n))$, y para algoritmos destino polinomiales no triviales esa composición puede crecer superpolinomialmente. Por tanto, el control polinomial de g_f es una hipótesis estructural adicional y no una consecuencia automática de la corrección semántica del puente.

Corolario 8.16 (No transportabilidad sobre una subfamilia exponencialmente expandida). *Supóngase que existe una familia infinita de instancias $x_n \in L_{R_0}$ con $|x_n|_{R_0} = n$ y un escalar $c > 1$ tales que*

$$|f(x_n)|_{R_1} \geq c^n \quad \text{para infinitos } n.$$

Supóngase además que existen $k \geq 1$ y $n_0 \in \mathbb{N}$ tales que el tiempo real de ejecución de A sobre esas instancias imagen satisface

$$\text{time}_A(f(x_n)) \geq |f(x_n)|_{R_1}^k \quad \forall n \geq n_0$$

dentro de la subfamilia considerada. Entonces el algoritmo inducido por el Teorema 8.13 no es polinomial.

Demostración. En la entrada x_n , el algoritmo inducido debe al menos computar $f(x_n)$ y ejecutar A sobre $f(x_n)$. En particular, su tiempo real de ejecución está acotado inferiormente por $\text{time}_A(f(x_n))$. Para n suficientemente grande dentro de la subfamilia infinita considerada, se tiene

$$\text{time}_A(f(x_n)) \geq |f(x_n)|_{R_1}^k \geq (c^n)^k = c^{kn}.$$

Esa cota inferior crece superpolinomialmente en n , luego el algoritmo inducido no puede ser polinomial. $\square$

Observación 8.17 (Separación conceptual). Los corolarios anteriores separan dos fenómenos: exactitud semántica y preservación de complejidad. Una codificación puede ser completamente correcta e incluso biyectiva, y aun así ser inútil desde el punto de vista algorítmico si el tamaño efectivo de la instancia se expande demasiado.

9. Esquema suficiente hacia $P = NP$ y localización de los huecos

Observación 9.1 (Motivación externa y papel lógico del presente manuscrito). El presente texto nace de una situación metamatemática precisa. En el prefacio de *Pedigree Polytopes* se afirma explícitamente que, puesto que la optimización lineal sobre el pedigree polytope $\text{conv}(P_n)$ sería equivalente al STSP, “this book establishes the theorem $NP = P$ ”; y en la hoja de ruta del capítulo 7 se sostiene que una verificación eficiente de membresía, combinada con un puente desde membresía a separación y desde allí a optimización, colocaría al problema objetivo en P y por tanto forzaría $P = NP$ [1, prefacio, pp. ix–xi]; el resumen programático del capítulo 1 reitera que el capítulo 7 usa esa cadena para concluir que un problema en la clase NP-completa pertenece a P y, como consecuencia inmediata, $NP = P$ [1, cap. 1, pp. 21–22]. El objetivo de la teoría presente no es negar informalmente ese programa, sino tiparlo y aislar exactamente qué piezas faltan para que una cadena de ese tipo sea un teorema válido.

Definición 9.2 (Compañero decisional de una familia de optimización). Un *compañero decisional* para la familia $\mathcal{P} = (S_n, C_n, \kappa_n)_{n \geq 1}$ es un lenguaje $L_{\mathcal{P}} \subseteq \{0, 1\}^*$ junto con un algoritmo polinomial que, dada una palabra $u \in \{0, 1\}^*$, produce un natural $n(u)$, un costo $c(u) \in C_{n(u)}$ y un umbral racional $\lambda(u) \in \mathbb{Q}$ tales que

$$u \in L_{\mathcal{P}} \iff \min_{s \in S_{n(u)}} \kappa_{n(u)}(c(u), s) \leq \lambda(u).$$

Definición 9.3 (Evaluabilidad polinomial del objetivo). Diremos que $\mathcal{P}$ tiene *objetivo polinomialmente evaluable* respecto de una codificación combinatoria $(\text{Enc}_n^S)_n$ si existe un algoritmo que, dada la entrada binaria $(n, c, \text{Enc}_n^S(s))$, calcula exactamente $\kappa_n(c, s)$ en tiempo polinomial en $\langle n, c \rangle + |\text{Enc}_n^S(s)|$.

Observación 9.4 (La membresía eficiente no cierra por sí sola el puente). Ningún resultado demostrado en este manuscrito permite inferir, a partir de un algoritmo de membresía para Q_n , la existencia de un algoritmo de optimización lineal sobre Q_n . Para cerrar ese paso hace falta un puente formal adicional en el modelo adoptado: una descripción explícita uniformemente generable en tiempo polinomial en $\langle n \rangle$ y de tamaño polinomial en $\langle n \rangle$, un oráculo de separación con control bit-complejo, o una reducción demostrada de membresía a separación/optimización válida con sus hipótesis geométricas y numéricas. Éste es exactamente uno de los puntos donde un programa tipo “membresía $\Rightarrow$ optimización $\Rightarrow P = NP$ ” debe ser sellado y no sólo enunciado.

Teorema 9.5 (Esquema suficiente para cerrar $P = NP$ a partir de una representación exacta). Sea $L_{\mathcal{P}}$ un compañero decisional de $\mathcal{P}$. Supóngase que se satisfacen simultáneamente las condiciones siguientes:

- (1) $L_{\mathcal{P}}$ es NP-completo bajo reducciones de Karp;
- (2) se ha fijado una codificación combinatoria $(\text{Enc}_n^S)_{n \geq 1}$ para $\mathcal{P}$;
- (3) $\mathcal{P}$ tiene objetivo polinomialmente evaluable respecto de $(\text{Enc}_n^S)_n$;
- (4) $\mathcal{P}$ admite una representación exacta

$$\mathcal{R} = (Q_n, D_n, \phi_n, \psi_n, \Lambda_n, \beta_n)_n$$

que es polinomialmente fiel y fuertemente exacta;

- (5) existe un algoritmo polinomial que, dada la entrada (n, w) con $w \in \mathbb{Q}^{d(n)}$, produce un punto óptimo racional

$$x^* \in Q_n \cap \mathbb{Q}^{d(n)}$$

de

$$\min_{x \in Q_n} w \cdot x,$$

o bien existe ya demostrado, en el modelo oracular o explícito adoptado, un puente formal desde la interfaz disponible para Q_n hasta un procedimiento polinomial que produzca tal punto óptimo racional.

Entonces $L_{\mathcal{P}} \in P$. En particular, si existe un compañero decisional $L_{\mathcal{P}}$ de $\mathcal{P}$ que sea NP-completo, entonces se concluye $P = NP$.

Demostración. Por las hipótesis (2)–(5) y el Teorema 5.3, la familia $\mathcal{P}$ se resuelve en tiempo polinomial: dada la instancia $c(u)$, la hipótesis (5) produce, para

$$w = \Lambda_{n(u)}(c(u)),$$

un punto óptimo racional de la relajación con el tipo exacto requerido por el Teorema 5.3. Aplicando ese teorema obtenemos en tiempo polinomial un código $\text{Enc}_{n(u)}^S(s^*)$ de una solución óptima $s^* \in S_{n(u)}$. Por la hipótesis (3), también en tiempo polinomial calculamos exactamente el valor

$$\kappa_{n(u)}(c(u), s^*) = \min_{s \in S_{n(u)}} \kappa_{n(u)}(c(u), s).$$

Comparando ese valor con el umbral racional $\lambda(u)$, decidimos si

$$\min_{s \in S_{n(u)}} \kappa_{n(u)}(c(u), s) \leq \lambda(u),$$

es decir, decidimos si $u \in L_{\mathcal{P}}$. Luego $L_{\mathcal{P}} \in P$. Si además $L_{\mathcal{P}}$ es NP-completo por la hipótesis (1), se concluye $P = NP$ por el criterio estándar de Cook–Karp. $\square$

Observación 9.6 (Estatus lógico de la hipótesis (5)). Cuando la cláusula alternativa de la hipótesis (5) se usa a través de un “puente formal” previamente demostrado, el Teorema 9.5 debe leerse como un teorema-esquema suficiente: una vez cerrado externamente ese puente en el modelo adoptado, la conclusión $L_{\mathcal{P}} \in P$ se sigue internamente por composición con el Teorema 5.3. Si, en cambio, se asume directamente la existencia del algoritmo que produce x^* , el teorema queda completamente interno.

Corolario 9.7 (Lectura formal del programa de los pedigree polytopes). *Para convertir una afirmación del tipo*

“optimizar sobre el pedigree polytope resuelve STSP y por tanto $P = NP$ ”

en un teorema válido dentro del marco presente, no basta una sola pieza parcial —por ejemplo, un procedimiento de membresía—. Hace falta cerrar toda la cadena del Teorema 9.5: representación exacta, fidelidad polinomial, soporte óptimo correcto, exactitud fuerte, interfaz formal desde membresía/separación hasta un procedimiento polinomial que produzca un punto óptimo racional de la relajación, y reducción decisional correctamente tipada.

Demostración. Es una reformulación directa del Teorema 9.5 aplicada a una familia objetivo como STSP. $\square$

10. Obstrucciones formales

Proposición 10.1 (Obstrucción por falla de representabilidad lineal). *Si para algún n el criterio lineal del Teorema 7.2 falla, entonces no existe levantamiento exacto lineal de costos en esa representación.*

Demostración. Es exactamente el contenido del Corolario 7.3. □

Proposición 10.2 (Obstrucción por falla de soporte óptimo). *Si existe $c \in C_n$ tal que*

$$\arg \min_{x \in Q_n} \Lambda_n(c) \cdot x \cap D_n = \emptyset,$$

entonces el soporte óptimo correcto falla. En particular, optimizar sobre Q_n por sí solo, o dentro del esquema presente sin un extractor o criterio adicional, no resuelve el problema original, aunque exista conservación exacta de valor sobre D_n .

Demostración. Si ningún óptimo de Q_n cae en D_n , entonces no hay garantía de que la relajación contenga siquiera un vértice representable óptimo. En consecuencia, falla la hipótesis estructural mínima que permite conectar el óptimo continuo con el óptimo discreto dentro del esquema formal desarrollado aquí; cualquier puente ulterior requeriría una capa adicional no incluida en la teoría presente. □

Observación 10.3 (La barrera de extracción es funcional, no decisional). La extracción

$$(c, x^*) \longmapsto s^*$$

es un problema funcional. Por ello, una barrera de complejidad para la exactitud fuerte no debe formularse aquí en términos de NP-dureza de Karp, que es una noción primaria para problemas de decisión. Para convertir esta intuición en un teorema negativo haría falta tipar formalmente la extracción en clases funcionales apropiadas —por ejemplo FP, FNP o una noción análoga de dureza para problemas de búsqueda— o bien pasar a una versión decisional asociada.

Proposición 10.4 (Obstrucción estructural a la rama explícita de fidelidad polinomial). *Si para infinitos n toda descripción racional explícita de Q_n tiene tamaño binario superpolinomial en $\langle n \rangle$, entonces la alternativa explícita del ítem (3a) de la Definición 4.4 no puede satisfacerse. En particular, cualquier demostración de fidelidad polinomial deberá pasar por una interfaz oracular o implícita.*

Demostración. La rama explícita de la Definición 4.4 exige que una descripción racional de Q_n pueda generarse uniformemente en tiempo polinomial en $\langle n \rangle$ y que la salida producida tenga tamaño también polinomial en $\langle n \rangle$. Si toda descripción explícita tiene tamaño superpolinomial en $\langle n \rangle$ para infinitos n , esa alternativa queda descartada por una barrera elemental de escritura de salida. □

Proposición 10.5 (Explosión binaria funcional en la capa de fidelidad). *Si no existe un polinomio p tal que*

$$\langle \Lambda_n(c) \rangle, \langle \beta_n(c) \rangle \leq p(\langle n, c \rangle) \quad \forall n, \forall c \in C_n,$$

entonces la fidelidad polinomial falla.

Demostración. La Definición 4.4 exige control polinomial uniforme sobre el tamaño binario de salida de las aplicaciones que pertenecen a la representación exacta, en particular Λ_n y β_n . Si ese control falla, falla la fidelidad polinomial por definición. □

Proposición 10.6 (Explosión binaria o temporal en la capa de extracción fuerte). *Si no existe un polinomio q tal que el costo y el tamaño binario de salida del extractor*

$$E_n(c, x^*)$$

queden acotados por

$$q(\langle n, c, x^* \rangle)$$

en su dominio natural, entonces falla la exactitud fuerte efectiva/polinomial. En particular, bajo esa falla no puede aplicarse el Teorema 5.3 para obtener resolución polinomial de la familia a través de dicho extractor.

Demostración. La Definición 4.6 exige precisamente que el extractor fuerte sea ejecutable en tiempo polinomial en $\langle n, c, x^* \rangle$. Si además se pretende integrarlo en un puente algorítmico polinomial, también debe mantenerse control polinomial del tamaño binario de su salida racional. Si alguna de esas dos cotas falla, no falla la fidelidad polinomial de la representación, sino la capa de exactitud fuerte efectiva necesaria para el Teorema 5.3. $\square$

Observación 10.7. La cuantificación correcta depende del tipo de dato. La descripción de Q_n es un objeto estructural indexado sólo por n ; $\Lambda_n(c)$ y $\beta_n(c)$ pertenecen a la capa de fidelidad polinomial de la representación; en cambio el extractor $E_n(c, x^*)$ pertenece a la capa adicional de exactitud fuerte efectiva y debe compararse con el tamaño binario completo de sus entradas naturales.

Observación 10.8 (No composicionalidad como observación y no como teorema). Puede ocurrir que existan dos puentes parciales

$$\mathcal{P} \rightsquigarrow R, \quad R \rightsquigarrow R',$$

pero que la compatibilidad de costos no haya sido formalizada de manera estable bajo composición. En ausencia de una noción explícita de composición de representaciones exactas, esta advertencia debe leerse como observación estructural y no como obstrucción demostrada.

11. Programa matemático y problemas abiertos

Los resultados anteriores cierran el núcleo mínimo del manuscrito. Lo que sigue debe quedar explícitamente como agenda formal.

Problema abierto 11.1 (Ley de composición). Definir una categoría, o al menos una clase cerrada bajo composición, de representaciones exactas y cambios de representación semánticamente fieles para optimización, de modo que pueda demostrarse cuándo la fidelidad y la preservación de óptimos se heredan por composición.

Problema abierto 11.2 (Extracción funcional tipada). Tipar formalmente el problema de extracción fuerte en clases funcionales apropiadas y estudiar barreras inferiores dentro de ese marco.

Problema abierto 11.3 (Criterio lineal implícito). Encontrar una formulación del criterio

$$K_n(C_n) \subseteq \text{Col}(M_n)$$

que no requiera enumerar S_n explícitamente y sea utilizable con oráculos de separación, de optimización o con realizaciones implícitas del operador B_n .

Problema abierto 11.4 (Compatibilidad con formulaciones extendidas). Relacionar el criterio lineal del Teorema 7.2 con nociones estándar de formulación extendida, extensión poliedral y complejidad de oráculos. La pregunta precisa es cuándo la existencia de una formulación extendida pequeña interactúa, o no, con la existencia de un levantamiento exacto de costos sobre D_n .

Problema abierto 11.5 (Instanciación en STSP). Para el TSP simétrico, construir explícitamente:

- (1) la aplicación ϕ_n entre tours y vértices representables;
- (2) el subconjunto $D_n \subseteq \text{Vert}(Q_n)$;
- (3) el levantamiento de costos Λ_n ;
- (4) la verificación del criterio lineal;
- (5) el soporte óptimo correcto;
- (6) un extractor polinomial de tours óptimos;
- (7) la compatibilidad con una formulación poliedral concreta o con un oráculo de separación efectivamente ejecutable.

La teoría sólo produce un puente completo una vez cerrados todos esos puntos.

Problema abierto 11.6 (Puente membresía–separación–optimización). Dentro de un modelo de complejidad bit-exacto y uniformemente tipado, precisar condiciones suficientes y necesarias bajo las cuales un procedimiento de membresía eficiente para una familia de politopos permite construir separación eficiente, y bajo cuáles de esas condiciones se obtiene optimización polinomial.

12. Conclusión

La presente regularización deja una tesis nítida y correctamente acotada:

Dentro del marco formal introducido aquí, una biyección entre objetos no basta. Para transferir el valor óptimo y la existencia de un óptimo representable se necesita una representación exacta con soporte óptimo correcto; para transferir efectivamente una solución a partir de un óptimo de la relajación se necesita exactitud fuerte, o una hipótesis equivalente de extracción; para transportar algoritmos entre representaciones se necesita además control uniforme del crecimiento representacional, inversión tipada de las codificaciones de solución y una decodificación polinomial correctamente cuantificada respecto del tamaño real de la salida.

En ese marco, el criterio lineal

$$K_n(C_n) \subseteq \text{Col}(M_n)$$

es el primer test estructural serio para decidir si existe o no un levantamiento exacto de costos. Pero el propio manuscrito deja igualmente claro el límite de ese criterio: por sí solo es algebraico; sólo se vuelve constructivo cuando se añade un mecanismo efectivo que evite la materialización exponencial.

La teoría queda así dividida en tres niveles complementarios. El primero, ya cerrado, es un núcleo irrefutable de conservación exacta, equivalencia afín uniformemente controlada, cambio

de representación correctamente tipado y existencia lineal. El segundo, también ya formalizado, es un esquema suficiente para convertir una representación exacta en un teorema del tipo $L \in P$, e incluso en $P = NP$, siempre que se cierren todas las hipótesis del Teorema 9.5. El tercero, todavía abierto, es el de la instanciación efectiva en familias concretas grandes —como STSP y los pedigree polytopes—, donde todavía deben sellarse el puente membresía/separación/optimización, la extracción fuerte concreta y la verificación completa de fidelidad polinomial.

Ésa es precisamente la frontera matemática real del programa: no una intuición geométrica aislada, sino la clausura simultánea de todos los puentes que llevan desde una representación alternativa hasta un teorema válido de complejidad.

Referencias

- [1] T. S. Arthanari, *Pedigree Polytopes: New Insights on Computational Complexity of Combinatorial Optimisation Problems*, Springer Nature Singapore, Singapore, 2023.
- [2] S. A. Cook, “The complexity of theorem-proving procedures,” in *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, ACM, 1971, pp. 151–158.
- [3] M. Grötschel, L. Lovász y A. Schrijver, *Geometric Algorithms and Combinatorial Optimization*, Algorithms and Combinatorics 2, Springer, Berlin, 1988.
- [4] R. M. Karp, “Reducibility among combinatorial problems,” in R. E. Miller y J. W. Thatcher (eds.), *Complexity of Computer Computations*, Plenum Press, New York, 1972, pp. 85–103.
- [5] J.-F. Maurras, “From membership to separation, a simple construction,” *Combinatorica* **22** (2002), no. 4, 531–536.